

LiDAR Intensity Preprocessing and Visualization Report

InvRGBL Repository Note

April 8, 2026

1 Purpose

This note documents how LiDAR intensity preprocessing is implemented in the current InvRGBL codebase and how the accompanying visualization script should be interpreted.

The goal of the preprocessing stage is to convert raw Waymo LiDAR returns into per-camera sparse intensity maps that can be consumed by the training pipeline. The goal of the visualization stage is to verify that these maps are geometrically aligned with the RGB images and numerically well-formed.

2 Files Involved

The current implementation is split across the following files:

- `datasets/waymo/waymo_preprocess.py`: extracts LiDAR points, projects them to each camera, and saves sparse intensity maps.
- `datasets/waymo/waymo_utils.py`: provides the camera projection utility `project_vehicle_to_image`.
- `tools/visualize_intensity.py`: generates debugging visualizations and summary statistics.
- `datasets/base/pixel_source.py`: loads the saved `.npy` intensity maps during training.

3 Preprocessing Pipeline

3.1 Input Data

The Waymo raw data consists of TFRecord files containing synchronized camera images, LiDAR range images, calibration parameters, vehicle pose, and annotations.

For intensity preprocessing, the implementation uses:

- the LiDAR range images from both returns,
- the per-frame vehicle pose,
- the camera calibration parameters,
- the LiDAR intensity values attached to each point.

3.2 Point Extraction

Inside `WaymoProcessor.save_intensity`, the code first converts the raw Waymo range images into point clouds using the existing helper:

```
convert_range_image_to_point_cloud_flow(..., ri_index=0)
convert_range_image_to_point_cloud_flow(..., ri_index=1)
```

The first and second returns are concatenated. For each point, the implementation keeps its 3D location in the vehicle frame and its associated LiDAR intensity.

If the two returns are denoted by

$$P^{(0)}, I^{(0)} \quad \text{and} \quad P^{(1)}, I^{(1)},$$

then the merged point set is

$$P = [P^{(0)}; P^{(1)}], \quad I = [I^{(0)}; I^{(1)}].$$

3.3 Projection to the Camera Plane

For each of the five Waymo cameras, the code calls:

```
project_vehicle_to_image(frame.pose, calib, points)
```

This function transforms points from the vehicle frame into world coordinates using the ego pose, then projects them into the image plane using the Waymo camera model and calibration metadata.

Each projected point yields

$$(u_i, v_i, o_i),$$

where u_i and v_i are image coordinates and $o_i \in \{0, 1\}$ indicates whether the projection is valid.

Only valid points are kept. Their coordinates are clipped to the image bounds:

$$u_i \leftarrow \text{clip}(u_i, 0, W - 1), \quad v_i \leftarrow \text{clip}(v_i, 0, H - 1).$$

3.4 Sparse Intensity Map Construction

For each camera, the implementation creates a zero-initialized image:

$$M \in \mathbb{R}^{H \times W}.$$

For every valid projected point, the corresponding pixel receives its LiDAR intensity:

$$M[v_i, u_i] = I_i.$$

This produces a sparse image because only a small subset of RGB pixels are directly observed by LiDAR. In road scenes, most nonzero values appear in the lower part of the frame, where the road and nearby objects lie.

3.5 Normalization

The current code normalizes each camera-frame intensity map independently before saving:

$$\hat{M} = \begin{cases} M / \max(M), & \text{if } \max(M) > 0, \\ M, & \text{otherwise.} \end{cases}$$

This makes all saved maps lie in the range $[0, 1]$. The normalization is per image, not global across the dataset.

3.6 Saved Output Format

Each intensity map is saved as a NumPy array with shape

$$H \times W \times 1$$

and type `float32`.

The output naming convention is:

```
data/waymo/processed/training/<scene_id>/intensity/<frame>_<camera>.npz
```

For example:

```
data/waymo/processed/training/023/intensity/000_0.npz
```

4 How the Training Loader Uses the Maps

The training pipeline loads these `.npz` files in `pixel_source.py`. Because the maps are sparse, the loader preserves the sparse structure when resizing them to the configured image resolution.

This is important: a standard dense interpolation would blur and distort sparse LiDAR support. Instead, the implementation converts the map to a sparse COO representation, rescales the occupied coordinates, and reconstructs the resized image.

5 Why Basic Heatmaps Look Wrong

Two facts make naive visualization misleading:

1. The maps are extremely sparse. Typically, more than 99% of pixels are zero.
2. The nonzero values are strongly skewed toward very small magnitudes. Most positive values can be on the order of 10^{-6} to 10^{-5} after normalization.

This means a plain call such as

```
plt.imshow(intensity_map)
```

often produces an image that looks almost entirely black or uniformly dark.

Even a naive logarithmic transform can fail, because for small x ,

$$\log(1 + x) \approx x.$$

So if x is already tiny, `log1p` still produces tiny values with almost no visible contrast.

6 Visualization Script

6.1 Script Purpose

The helper script

```
python tools/visualize_intensity.py --scene 23 --frame 0 --camera 0
```

produces a small debugging package for one scene, frame, and camera.

Its outputs are written to:

```
debug/intensity_vis/<scene>/<frame_camera>/
```

6.2 Outputs

The script saves the following files:

File	Purpose
<code>overlay.png</code>	RGB image with projected LiDAR support drawn in red
<code>mask.png</code>	binary mask of pixels that received LiDAR intensity
<code>nonzero.png</code>	intensity image with zero background hidden
<code>log.png</code>	positive-pixel log-scaled visualization
<code>lognorm.png</code>	logarithmic colormap normalization on expanded intensity

6.3 Visibility Expansion for Debugging

Because valid LiDAR returns are often only one pixel wide, the script dilates them for display only. This does not modify the stored data. It only makes the saved images easier to read.

If the original support mask is B , the script computes an expanded mask $\tilde{B}$ using a small neighborhood max operation over a few iterations. The same idea is applied to the intensity values for visualization-only images.

This is why the overlay and the contrast-enhanced images show clear scanlines instead of nearly invisible isolated pixels.

6.4 How to Interpret Each Visualization

Overlay. This is the most important diagnostic. If the preprocessing is correct, the projected support should align with road surfaces, nearby vehicles, curbs, and the lower parts of objects. It should not heavily populate the sky.

Mask. This shows where LiDAR support exists, independent of intensity magnitude. It is useful for checking coverage and calibration.

Nonzero map. This hides the zero background entirely and colors only valid LiDAR returns. It is often the clearest view of the structure of the map.

Log map. This shows the same nonzero support after logarithmic scaling and contrast normalization. It is useful for inspecting relative strength differences among the visible returns.

7 Observed Validation Behavior

For a tested sample scene and camera, the projected intensity map showed:

- roughly twenty-four thousand nonzero pixels,
- support spanning almost the full image width,
- vertical support concentrated in the lower part of the image,
- scanline-like horizontal bands consistent with LiDAR acquisition,
- good spatial alignment with the RGB image after overlay.

These are the expected qualitative signatures of a correct projection-based LiDAR intensity map in a forward-facing urban driving scene.

8 How to Run the Preprocessing

To generate intensity maps during Waymo preprocessing, the current command is:

```
python datasets/preprocess.py \  
  --data_root data/waymo/raw/ \  
  --target_dir data/waymo/processed \  
  --dataset waymo \  
  --split training \  
  --scene_ids 23 114 327 621 703 172 552 788 \  
  --workers 8 \  
  --process_keys intensity
```

Note that the output scene folders correspond to the indices used by the preprocessing pipeline.

9 How to Run the Visualization

Example:

```
python tools/visualize_intensity.py --scene 23 --frame 0 --camera 0
```

This generates a report directory under:

```
debug/intensity_vis/023/000_0/
```

10 Recommended Validation Checklist

To confirm the preprocessing is working correctly:

1. Check that **intensity** folders are created under each processed scene.
2. Verify that the saved arrays have shape $H \times W \times 1$ and type **float32**.
3. Confirm that the maps contain nonzero pixels but remain highly sparse.
4. Inspect **overlay.png** to verify geometric alignment with the RGB image.
5. Inspect **nonzero.png** or **log.png** to verify the expected scanline structure.
6. Optionally compare support against projected depth maps for stricter validation.

11 Conclusion

The implemented LiDAR intensity preprocessing in this repository follows a straightforward projection-based design:

1. extract points and intensities from Waymo LiDAR returns,
2. project the points into each camera,

3. rasterize them into sparse per-camera intensity maps,
4. normalize and save them as `.npy` files,
5. verify them with overlay-based and contrast-enhanced visual diagnostics.

The most reliable qualitative check is the RGB overlay. The contrast-enhanced views are useful supporting diagnostics, but plain heatmaps alone are not sufficient because of sparsity and heavy value skew.